

Category B Practice Problems

PROBLEM 1 • Basic

The Talking Toy Box

A toy store app wants every toy to make its own sound, but the exact sound is different for every kind of toy — there's no such thing as a generic toy with no specific sound of its own.

Requirements

- Toy must be an abstract class with an abstract method `makeSound()`, and a final `toyId` field assigned via a shared static counter inside its constructor.
- `ToyCar` and `ToyRobot` must both extend `Toy` directly and implement `makeSound()` with genuinely different sounds.
- Confirm, through your design, that `new Toy()` cannot compile — only a specific toy can ever be created.

Function Signature(s)

```
public abstract class Toy // assigns a final toyId internally
public abstract String makeSound();
String getToyId()
public ToyCar(String name)
public ToyRobot(String name)
```

Examples

Input	Output
<code>ToyCar c = new ToyCar("Speedster");</code> <code>c.makeSound()</code>	"Speedster: Vroom vroom!"

Input	Output
<code>ToyRobot r = new ToyRobot("Bolt");</code> <code>r.makeSound()</code>	"Bolt: Beep boop!"

Input	Output
<code>c.getToyId()</code> <code>r.getToyId()</code>	"TOY-1001" "TOY-1002"

Explanation: The static counter advances once per `Toy` object regardless of which subclass actually constructed it.

HINT Put the static counter increment inside `Toy`'s own constructor — every subclass reaches it through `super()`, so it only needs to exist in one place.

CONSTRAINTS: name is never blank.

PROBLEM 2 • Basic**Warehouse Label Printer**

A warehouse system wants any item that needs a printed label — packages and invoices alike — to support the same printing action, even though packages and invoices have nothing else in common.

Requirements

- Define an interface `Printable` with a method `printLabel()` returning a `String`.
- `PackageBox` and `Invoice` must both implement `Printable` directly, with no relationship to each other or to any shared parent class.
- Write a static method `printAll(Printable[] items)` that loops through a mixed array and calls `printLabel()` on each.

Function Signature(s)

```
public interface Printable { String printLabel(); }  
public PackageBox(String trackingId)  
public Invoice(String invoiceNumber)  
static void printAll(Printable[] items)
```

Examples

Input	Output
<pre>PackageBox p = new PackageBox("TRK-88"); p.printLabel()</pre>	<pre>"Package label: TRK-88"</pre>

Input	Output
<pre>Invoice i = new Invoice("INV-42"); i.printLabel()</pre>	<pre>"Invoice label: INV-42"</pre>

Input	Output
<pre>printAll(new Printable[]{ p, i })</pre>	<pre>"Package label: TRK-88" "Invoice label: INV-42"</pre>

HINT Since `PackageBox` and `Invoice` share nothing else, `Printable` is the only thing that lets one method work on both.

CONSTRAINTS: `trackingId` and `invoiceNumber` are never blank.

PROBLEM 3 • Intermediate**Orchestra Warm-Up Routine**

An orchestra app models instruments as a family: every Instrument can be played, and a Violin is a string instrument with its own extra warm-up step layered on top of the general string-instrument routine.

Requirements

- Violin must extend StringInstrument, and StringInstrument must extend Instrument — a genuine multilevel hierarchy three classes deep, with each subclass constructor calling its parent's constructor using `super(...)`.
- Instrument must be abstract with an abstract `play()` method; StringInstrument must override `play()` with its own concrete version; Violin must override `play()` again.
- Each override (in StringInstrument and Violin) must call `super.play()` first and then add its own extra detail, rather than rebuilding the whole message from scratch.

Function Signature(s)

```
public abstract class Instrument()
public abstract String play();
public StringInstrument() // overrides play(), calls super.play()
public Violin() // overrides play() again, calls super.play()
```

Examples

Input	Output
StringInstrument s = new StringInstrument(); s.play()	"Strumming the strings"

Input	Output
Violin v = new Violin(); v.play()	"Strumming the strings, with a bow drawn across four strings"

Explanation: Violin reuses StringInstrument's own message via `super.play()` and appends its own detail, rather than restating the whole thing itself.

HINT Each `play()` override should call `super.play()` first, then add only the new detail that level introduces.

CONSTRAINTS: No arguments are required to construct any of the three classes.

PROBLEM 4 • Intermediate

Smart Kitchen Assistant

A smart kitchen app manages tools that prepare food, and separately, some tools also need to report that they've been cleaned — a completely different concern from how they prepare food.

Requirements

- KitchenTool must be an abstract class with an abstract method prepare(), and a private speedLevel field exposed only through JavaBean getSpeedLevel()/setSpeedLevel() methods — setSpeedLevel() must reject any value outside 1-5.
- Define an interface Washable with a method clean() returning a String.
- Blender must extend KitchenTool and implement Washable.

Function Signature(s)

```
public abstract class KitchenTool()  
public abstract String prepare();  
int getSpeedLevel()  
void setSpeedLevel(int speedLevel)  
public interface Washable { String clean(); }  
public Blender()
```

Examples

Input	Output
Blender b = new Blender(); b.setSpeedLevel(3); b.getSpeedLevel()	3

Input	Output
b.setSpeedLevel(9)	rejected, speed level stays 3

Input	Output
b.prepare() b.clean()	"Blending at speed 3" "Blender rinsed and dried"

HINT setSpeedLevel() should check the range before assigning anything — reject first, assign second.

CONSTRAINTS: speedLevel must be between 1 and 5 to be accepted.

PROBLEM 5 • Intermediate

Package Drop-Off Log

A courier app logs every delivery note, but confirming delivery sometimes includes a signature and sometimes doesn't — the confirmation method should work either way.

Requirements

- DeliveryNote must be an abstract class with an abstract confirmDelivery() method, plus an overloaded confirmDelivery(String signature) — true compile-time polymorphism.
- ParcelNote and LetterNote must both extend DeliveryNote directly and implement confirmDelivery() with genuinely different messages.
- Write a static method logAll(DeliveryNote[] notes) that loops through a mixed array and calls confirmDelivery() on each polymorphically.

Function Signature(s)

```
public abstract class DeliveryNote()
public abstract String confirmDelivery();
String confirmDelivery(String signature)
public ParcelNote(String trackingId)
public LetterNote(String trackingId)
static void logAll(DeliveryNote[] notes)
```

Examples

Input	Output
ParcelNote p = new ParcelNote("TRK-1"); p.confirmDelivery()	"Parcel TRK-1 delivered"

Input	Output
<code>p.confirmDelivery("J. Smith")</code>	"Parcel TRK-1 delivered, signed by J. Smith"

Input	Output
<pre>DeliveryNote ref = p; // upcasting: ParcelNote stored as its parent type logAll(new DeliveryNote[]{ ref, new LetterNote("TRK 2") })</pre>	<pre>"Parcel TRK-1 delivered" "Letter TRK-2 delivered"</pre>

HINT Let `confirmDelivery(String signature)` call `confirmDelivery()` internally and just add the signature detail on top — that way the core message only ever lives in one place.

CONSTRAINTS: `trackingId` is never blank.